MemDB2 Codebase Review: Fresh-Slate Summary

Scope: all 30 units in Codebase_for_claude.zip (about 24k lines excluding the test form): MemDB2 (core, API, buffering, indexing, journal, streaming), IndexedStore, BinaryTree, CoWTree, Reffed, Trackables, RWWLock/TinyLock/StripedLock/SRWLock, DLList/DLThreadQueue, Parallelizer, BufferedFileStream, NullStream, CRC32, PComp, PatchFreeMem. Line numbers refer to the source files. Release builds have **assertions disabled** (DCC_AssertionsAtRuntime=false), so every Assert-only check below is absent in production. 'Latent' means the defect exists but no current MemDB2 code path reaches it.

Category	High	Medium	Low / Low-Med	Total
1. Bugs	2	7	13	22
2. Optimisations	2	4	8	14
3. Not worth fixing (already handled)	-	-	-	17

Fix first: B1 (failed fsync silently ignored in release), B2 (InitDB retry deadlock), B3/B4 (checkpoint durability and unbounded disk growth), B5 (journal shutdown race), then O1/O2 (thread-per-entity parallelism and file-per-commit journal), which are the biggest scalability limits.

1. Bugs

ID	Severity	Location	Problem	Suggested fix
B1	**High**	MemDB2Journal.pas ~1000-1011 (PerformMultiTransactionWrite)	FlushFileBuffers / fsync result is only checked with Assert(FlushOK). Release config sets DCC_AssertionsAtRuntime=false, so a failed flush is silently ignored and an amFlushBuffers transaction is reported durable.	if not FlushOK then RaiseLastOSError (lets the existing except path call DoJournalError).
B2	**High**	MemDB2Journal.pas 573-582 (Initialise) + MemDB2.pas InitDB	Retrying InitDB after a failed initial load hangs forever. The journal thread has already left its loop (FState=jscFinished) but FJournalThread is still assigned, so no new thread is started; the jatInitialLoad action is queued with nobody to process it and InitDB waits on FInitWait (INFINITE).	In Initialise: if Assigned(FJournalThread) and FJournalThread.Finished then WaitFor + Free + nil before recreating.
B3	**Medium**	MemDB2Journal.pas 1044 (PerformCheckpoint)	Checkpoint .dbinit files are never flushed to disk (no FlushFileBuffers), and the loader always picks the newest .dbinit with no fallback. A power loss shortly after a checkpoint can leave a torn newest checkpoint, and the DB then refuses to load even though the older checkpoint + increments were intact.	Flush checkpoint file (and on POSIX the directory); write to temp name then rename; or fall back to the previous .dbinit if replay of newest fails.
B4	**Medium**	MemDB2Journal.pas (CleanupUnusedFiles only called from PerformInitialLoad)	Superseded journal files are only deleted at start-up. Runtime checkpoints add a full DB copy each time and never reclaim older .dbinit/.dbincr files, so disk use grows without bound during uptime.	After a checkpoint is durably written, delete files with seq < checkpoint seq.
B5	**Medium**	MemDB2Journal.pas 318-331, 466-534	Shutdown use-after-free race: DoJournalFinished sets FreeOnTerminate, nils FJournalThread, then fires OnJournalFinished, which unblocks StopDB. The thread then still runs Action.Free and re-reads Self.FState in the while condition, and the destructor no longer waits, so the owner can free the journal while the thread is still touching it.	Leave the loop immediately after jatLastAction via a local flag, or keep the thread ref and WaitFor in the destructor.
B6	**Medium**	MemDB2Streamable.pas 1199-1256, 484-500, 827-857	All deserialisation uses Stream.Read without checking the byte count. Truncated or failed reads leave locals uninitialised (ExpectTag's local tag can match by chance), and lengths are unvalidated (huge SetLength/GetMem). Real I/O errors from TReadOnlyCachedFileStream (FLastError never surfaced) get reported as "Corrupted input file".	Use ReadBuffer (raises EReadError); validate lengths against remaining stream size; surface FLastError.
B7	**Medium**	IndexedStore.pas 613, 797, 1182 (AddIndexNodeForItem / AsyncIndexAddWorker / AddItemInPlace) and _AddIndex	Rollback runs only for EOutOfMemory. Any other exception from CompareItems (e.g. EMemDBInternalException from CompareFields type mismatch in lookaside nodes) leaves an item in FItemRecList partly indexed with FCount not incremented, or a half-built index registered. On the async path the tail then frees the TSIndex, leaving item links pointing at freed memory.	Catch Exception (not just EOutOfMemory) for rollback; re-raise.
B8	**Medium**	RWWLock.pas 264	Batch end uses '=': if AdmittedContended = FRatios[Reason]. If SetRatio lowers a ratio below the current admitted count during a contended batch, the batch never ends, so other lock modes starve indefinitely.	Use >=.
B9	**Medium**	BufferedFileStream.pas 252-258 (SetSize)	Seek only updates FPosition (no SetFilePointer), so SetEndOfFile truncates or extends at whatever the OS pointer happens to be. The write cache isn't flushed and FFileSize/caches aren't updated. This is latent: MemDB2 doesn't call Size := today.	FlushCache; FileSeek(NewSize); SetEndOfFile; update FFileSize, invalidate read cache.
B10	**Low/Med**	IndexedStore.pas 1021 (PerformAsyncActions)	If ExecParallel raises (class not in the local filter: EAssertionFailed, custom, EMemDB*), the cleanup tail never runs. Indexes stay async and a deleting index's links are already freed, so ForceClearAsyncFlags then exposes dangling FIndexLink pointers. MemDB2 doesn't currently use the async path.	Wrap ExecParallel in try/finally that runs the tail; record all exceptions.
B11	**Low/Med**	IndexedStore.pas 1657 (DeleteChildren)	If any index is async-pending, RemoveItem returns rvAsyncInProgress without removing anything. GetAnItem returns the same record, so the loop never ends and Obj.Free is called repeatedly on the same object (double free).	Check RemoveItem result; break/raise.
B12	**Low**	RWWLock.pas 234 / 412 vs Destroy	The last thread signals FDestroyed inside FCrit and then calls FCrit.Release, while the destructor may already have woken and freed FCrit (use-after-free).	Signal after leaving FCrit, or have the destructor Acquire/Release FCrit before freeing.
B13	**Low**	MemDB2Streamable.pas 818 / 848-849	Blob size is UInt64 but TStream.Read/Write take a Longint count. Blobs of 2 GiB or more are silently written truncated (or not at all) while the size header claims the full size, which leaves the journal unloadable.	Reject size > MaxInt at WriteField, or chunk the I/O.

ID	Severity	Location	Problem	Suggested fix
B14	Low	MemDB2Streamable.pas 860-869, 484-500	TMemFieldData.FromStream frees blob Data on error but doesn't nil it (latent double free in the destructor). RecFromStream tests Rec.Data, whose variant storage overlaps the integer fields. List FromStream leaks NewItem on failure.	Nil after FreeMem; Release NewItem in an except block.
B15	Low	IndexedStore.pas 1895 (TIndexedStoreO.IndexInfoByOrdinal)	Checks Assigned(Tag) on the var output before it is populated, and FAllowNullTags has no setter, so the call fails whenever the caller nil-initialises Tag.	Drop the pre-check or check the result.
B16	Low	Trackables.pas 106	Under USE_TRACKABLES, TTrackedMemoryStream derives from TTrackProxy, not TMemoryStream (and proxies itself), so it isn't a stream in debug builds. Currently unused.	class(TMemoryStream), like TTrackedFileStream.
B17	Low	NullStream.pas 72-75	Write returns 0, so WriteBuffer/CopyFrom raise EWriteError, contrary to the 'bit bucket' intent. MemDB2 special-cases 'is TNullStream', so this is latent.	Return Count.
B18	Low	MemDB2.pas 1126 (InitDB)	Compares the un-normalised RootLocation with BaseDirectory (which has a trailing slash appended). A second InitDB with the same path and no trailing slash raises S_REINIT_DIFF_LOCATION.	AppendTrailingDirSlash(RootLocation) at the top of InitDB.
B19	Low	MemDB2Journal.pas 546	Destroying a TMemDB that was never initialised trips Assert(FState = jscFinished) (the state is jscCreated) in debug builds, raising from the destructor.	Accept jscCreated.
B20	Low	MemDB2Journal.pas 705-735, 873	GetInitialFileList: an early exit skips FindClose and leaks the DBInfo objects (RList.Free doesn't free items); StrToInt64 accepts negative/hex names. LastFileBlank is never set, so the blank-tail deletion is dead code.	try/finally FindClose + FreeRList; validate names; set or remove LastFileBlank.
B21	Low	MemDB2Buffered.pas 7723; MemDB2Indexing TakeNodesFrom	ConsolidateAndFreeCaches sizes only the node cache, so the pin cache still allocates during TakeNodesFrom. An out-of-memory error mid-copy leaves pointers in both caches, and they are later double-released.	Pre-size the pin cache too; move pointers then clear the source atomically.
B22	Low	Reffed.pas 293, 328	TTrackedReffed.GetExtraInfoText appends to an uninitialised Result; CloneByRef leaks the new list and its refs if AddRef raises.	result := 'Refcount: '...; try/except release in CloneByRef.

2. Optimisations

ID	Impact	Location	Issue	Suggestion
O1	**High**	Parallelizer.pas; MemDB2Buffered PreCommit/Commit/Rollback/index builds	ExecParallel creates and destroys one OS thread per task on every call, and MemDB2 queues one task per entity per phase (FEntityThrottle only limits concurrency, not thread count). Hundreds of tables means hundreds of thread creations per commit phase. A CreateThread failure during ccpData commit leaves entities partially committed and forces the DB into its error state.	Persistent pool of MaxParallel workers with a work queue; run inline when there's a single task.
O2	**High**	MemDB2Journal.pas (PerformMultiTransactionWrite)	Every write batch, which means every amFlushBuffers commit, creates a new .dbincr file, and there's no runtime rollover or auto-checkpoint. That produces millions of files per day, directory bloat, and slow start-up (FindFirst, sort, then open/close per file).	Append to a rolling segment file with per-record length + CRC32 framing (CRC32 unit already present). This also enables safe truncated-tail recovery.
O3	**Medium**	MemDB2.pas Checkpoint + Journal PerformCheckpoint	Full DB is serialised to a temp file, then the journal thread copies it to the .dbinit, so I/O and disk use are doubled.	Serialise straight to the final file (temp name + rename), or hand the temp file over by rename.
O4	**Medium**	MemDB2BufBase.pas 1070 (RequestChangeInternal)	DeepClone of the whole row/metadata happens while holding the per-object TTinyLock (spin + Sleep(0)). Allocation under a spinlock makes contending threads burn CPU.	AddRef Cur.Item, clone outside the lock, re-lock and verify Cur unchanged before installing.
O5	**Medium**	MemDB2Api.pas 1695 (API_DataFindEdgeByIndex)	Walks duplicates one node at a time (O(k)), with two format checks and field compares per step.	Add lower/upper-bound search to CoWTree (comparator treats equal as greater/less), giving O(log n).
O6	**Medium**	MemDB2Journal.pas 468	After each action the thread waits up to 5 s (poll) even when the queue is non-empty, unless a high-priority action is pending, which delays checkpoints and lazy batches.	Only wait when the queue is empty.
O7	**Low/Med**	MemDB2Buffered.pas 3801 (UserMoveToRowByIndexRoot)	Without an index (below snapshot isolation), skipping deleted/unpinnable rows is linear while holding FMasterRowLock, which blocks inserts (acknowledged TODO).	Bounded batches; drop/re-take the lock; or periodic compaction.
O8	**Low/Med**	StripedLock.pas; MemDB2BufBase TMemDbMultiBufferedStriped	Each row heap-allocates a TStripedLock object just to hold a Word (a TTrackable in debug builds). 65,536 TCriticalSections are created at unit init (MBs of memory, packed together so they false-share).	Inline the key in the row; fewer cache-line-padded stripes; lazy init.
O9	**Low**	BufferedFileStream.pas Read/Write	Requests larger than the cache are double-copied through it. TWriteCachedFileStream's read side defaults to a 16 KB cache, so journal CopyFrom of changesets/checkpoints runs through 16 KB.	Bypass the cache for large requests; pass FILE_CACHE_SIZE for the read stream.
O10	**Low**	MemDB2Buffered.pas 1109 (ParallelAddHandler)	Grows five dynamic arrays by one element per call, so copying is quadratic.	Pre-size to EList.Count.
O11	**Low**	MemDB2Buffered.pas 7809 (EntitiesByName)	Linear scan, pinning each entity's metadata, on every API name lookup (CreateTable, GetAPIObjectFromEntity, 4x in each FK pre-commit).	Per-transaction name-to-entity map.
O12	**Low**	MemDB2Misc.pas 768	NeverSleepOnMMThreadContention := true is forced globally at unit init and changes memory-manager behaviour for the whole host app (CPU spin).	Make it opt-in.
O13	**Low**	TinyLock.pas	Fixed 2000-spin then Sleep(0): no backoff, and Sleep(0) won't yield to lower-priority threads (priority-inversion risk).	Exponential backoff, SwitchToThread / Sleep(1) fallback.
O14	**Low**	CRC32.pas; StripedLock.Init	A byte-at-a-time CRC is used only to hash already-random GUIDs into stripes.	XOR-fold the GUID words; use slice-by-8 if CRC framing is adopted (O2).

3. Not worth fixing: already covered by error handling, rollback or exceptions

ID	Area	Why it's acceptable as-is
S1	MemDB2Api UserCommitCycle / UserRollbackCycle	Fail-stop design: pre-commit failures roll back index Next under the commit lock; anything after pre-commit, or a journalling failure, sets the DB error flag, which is checked synchronously under CommitLock, so memory and journal can't silently diverge. Rollback swallows the exception but flags the DB.
S2	MemDB2Journal AmalgamateTransactions / PerformMultiTransactionWrite	Every path, including exceptions, releases transactions, signals FlushFinishedEvent and balances HighPriorityDone; errors move the journal to its error state rather than killing the thread.
S3	TWriteCachedFileStream destructor flush failure	The exception propagates out of Free into PerformMultiTransactionWrite's handler and becomes DoJournalError; the handle is still closed in finally. Only the object memory leaks.
S4	BufferedFileStream constructor failure	CreateHandle raises before FHandle is assigned, FreeMem(nil) is safe, and FlushCache guards Assigned(FCache), so destruction of a partly-built object is clean.
S5	Parallelizer thread-creation failure mid-loop	finally frees (and joins) the threads already started; unstarted slots are nil. Re-raised exceptions keep the message but lose the subclass (e.g. EDivByZero becomes EIntError), which is acceptable.
S6	CoWTree insert/delete/rebalance	Exception-safe path copying: temps are released on exception and the 'consumed' flag disambiguates ownership of the supplied node. Release on nil is a no-op.
S7	TReffed TryAddRef / TryReleaseInt	CAS loop refuses to resurrect a zero refcount, so teardown races are detected and turned into an exception rather than a use-after-free.
S8	RemoveTransaction AsyncError rollback (goto txion_remove)	Skips the rollback cycle (pins leak), but only once the DB is already in its error state and must be restarted. Acknowledged TODO.
S9	Truncated final journal file	Load is refused with a clear error and no data is silently corrupted. Recovery would be nicer (see O2) but this behaviour is safe.
S10	GetAPIObject / PutAPIObjectLocal AddRef/Release window	Runs outside the lock, but entity proxies are also held by entity lists, so the refcount can't reach zero in that window (documented).
S11	PatchFreeMem	Only on XE4 Win64; byte-compares before patching, restores page protection in finally; returns false (ignored) on any mismatch.
S12	TMemDBDatabasePersistent / TMemDBTablePersistent release-mode destructors	Brute-force 'while TryRelease' loops run only at teardown with no clients; CHECK_TEARDOWN_REFS builds do the proper cleardown transaction.
S13	OOM-only leaks (StartTransaction list add, API_CreateTable/CreateIndex, GetAPIObject, CloneByRef)	Reachable only after allocation failure, where the process state is already compromised; documented as 'very unlikely'.
S14	Checkpoint holds CommitLock for the full serialise	Deliberate simplicity/safety trade-off, documented in source; commits block but stay consistent.
S15	TDLThreadQueue FDebugLocked shared flag	Debug-only check that another thread's lock can satisfy; no effect on release behaviour.
S16	TBinTree delete-by-CopyFrom node identity swap	Handled correctly for IndexedStore via TIndexNode.CopyFrom / IndexCopiedFrom re-linking; Trackables only uses internal items.
S17	FK pre-commit renamed table/index lookups returning nil	Nil results are caught by the 'same object' / AbsIdx checks, which raise before any dereference.